OS Lab

Users and Groups Management

Introduction

When we install a Linux distribution, we create a user by giving it a username and password.

When we open our terminal, we're logged in as our default user and any session in our terminal is usually started in the **Home directory (~)** of our user, which is located in /home/ directory.

In this presentation, we'll learn users and groups management in Linux.

We shall see how to; create new users, delete existing users, create new groups, delete existing groups, add users to groups, remove users from groups, make modifications to groups, make modifications to users

root user & its privileges

root user is the most powerful user with all rights and privileges (works as an **Administrator**)

It is created automatically when a linux environment is setup on a machine

We can login as the root user from our terminal and perform highly privileged activities which are not allowed for other users

Examples of privileged activities: adding or deleting a user account, adding or deleting a group, resetting password of a normal user, changing rights and privileges of any user, installing packages and utilities in the system ….. etc etc

root user & its privileges (….)

Examples of privileged activities: adding or deleting a user account, adding or deleting a group, logging in as another user without knowing its password, resetting password of another user, changing rights and privileges of other users, installing packages and utilities in the system, ….. etc etc

All commands relating to users & groups management are usually considered highly privileged commands and the root user can simply execute those commands without seeking permission/validation from any other entity

Examples: adduser, useradd, deluser, userdel, addgroup, groupadd, delgroup, groupdel, groupmod, usermod, visudo

sudo (group)

In linux, we have a facility to create and modify **groups** of users. (More on it later ….)

A lot of **groups** are created automatically when a linux environment is setup, among them is a group named named **sudo,** our default user is also added to the sudo group automatically

sudo group is a really powerful group and its members can perform most of the privileged activities on the system but …

root user has every right to restrict any permissions and privileges of the sudo group

sudo (command)

Members of sudo group can exercise their rights and perform highly privileged activities by typing **sudo** before the privileged command ;

sudo = do as a super user

e.g; **sudo su - root**

The above mentioned command is a highly privileged activity and it can be used to switch your user and login as the **root** user

In beginner friendly linux distros, this mechanism helps us to login as a root user without remembering any password for the root user and it is kept to facilitate us and helps us in accessing root user's terminal in recovery mode but we can definitely set a password for the root user and can even modify permissions of the sudo group to not allow them to switch to the root user itself.

sudo (command) …….

Some of the other privileged commands include ;

adduser, useradd, deluser, userdel, addgroup, groupdel, usermod, groupmod

all these commands are used to create,delete and modify user accounts and groups in a linux environment and members of sudo group can simply execute them by adding a sudo keyword before them. They will just be prompted for their own password and they are good to go.

e.g ; **sudo adduser newusername , sudo addgroup newgroupname … etc**

Creating a new user

We create a normal user account at the time of installation and similarly root user's account is automatically created with the installation.

A lot of **system accounts** are also created which are used by our OS to manage certain system level activities but we don't have much concern with those accounts (at least for now).

Now, we'll see how to **create new user accounts** which can be logged in and used to perform any normal activities as a normal user.

We have a command **useradd** for adding a user but it expects us to use a lot of flags/switches for doing very obvious configurations for the user

We can rather simply use **adduser** command to create a new user

e.g; **sudo adduser ali-hafeez**

This command will simply create a user named ali-hafeez and will also make a lot of default configurations for this user (details on next slide)

Note : root user doesn't need to use sudo with its privileged commands, it's just a compulsion for the members of the sudo group to use sudo keyword here.

Creating a new user (adduser command)

adduser is a high level command and it automatically takes care of all necessary configurations and entries for the user. We don't need to use any flags/switches for these obvious configurations;

setting **new password** for the user

setting the default shell of the user to **/usr/bin/bash**

creating a home directory for the user named after it in /home/ directory

assigning uid and gid to the user

creating a **primary group** named after the user and even adding the user to few **supplemental groups (more on primary vs supplemental groups later …)**

creating entries for the user and its primary group in **/etc/passwd , /etc/group** file

This command itself manages all these things and we are good to go with a ready to use user

/etc/passwd

In our directory system, there resides a file in the /etc/ directory and usually called as **passwd** file.

If we simply **display** the contents of that file we can see that it contains detailed entries of all users and system accounts on our machine.

This file contains exactly 7 fields for each account's entry and each field is simply separated by a colon (:)

i) username, ii) a field for password, iii) uid, iv) gid, v) Full-Name,Additional Info, vi) user's home directory path , vii) path of the default shell used for the user

We can use text editors to make changings to our user's accounts configurations from this file but it's not considered a good practice and we have better ways of modifying our user accounts (More on it later)

/etc/shadow

In /etc directory , there’s a file named **shadow** which serves as a database for storing encrypted hashes for passwords of all users

Previously, these hashes used to be kept in **/etc/passwd** along with other fields of the user but in modern linux systems they’re kept in a seperate file for adding another layer of security to the user’s privacy

Deleting a user (deluser command)

We can simply use **userdel** command for deleting a user.

Similarly, we have a much more powerful and easy-going command **deluser** for deleting a user and we can use **--remove-all-files** flag to remove all files of the user from our system, it literally removes everything relating to this user, all files and all entries of this user from our system are removed with this command

sudo deluser --remove-all-files username

Note : root user doesn’t need to use sudo with its privileged commands, it's just a compulsion for the members of the sudo group to use sudo keyword here.

Groups in Linux

In linux, we have a facility to create and modify **groups** of users.

A user can be a part of multiple groups and similarly each group can contain multiple users members

A lot of groups are created automatically for system accounts which are used for managing and sustaining system level activities but we don’t have to bother with most of them.

There are 2 types of groups for a user ;

Primary Group vs Supplemental/Extra Group

Groups in Linux (Primary vs Supplemental)

When a new user is created, a group named after its username is created and its assigned as the **primary group** for that user

It's true for all user accounts as well as the system accounts (which we talked about in previous slides)

The **primary group** of the root user is root

The primary group of our default normal user is usually named after its username

e.g ; If we create a new user with a username **ali-hafeez**, its **primary group** will be created with the name **ali-hafeez**

The user can be part of other groups as well, these groups are deemed as **supplemental groups** of that user

e.g ; **ali-hafeez** is also member of **sudo** group, so sudo group is his **supplemental group**

A user can have multiple supplemental/extra groups but a user only has 1 primary group

The primary group of a user can also be changed and we shall definitely see it in later on.

Groups in Linux (Creating a new group)

We can add/create new groups by simply using **groupadd** or **addgroup** command

We can simply give the groupname and the group will be created

e.g; **sudo addgroup section-b**

The above command will simply create a group named section-b and it'll also assign that group a **GID**

Note : root user doesn't need to use sudo with its privileged commands, it's just a compulsion for the members of the sudo group to use sudo keyword here.

Groups in Linux （ Deleting a group ）

we can delete/remove a group by simply using **groupdel** or **delgroup** command

We can simply give the groupname and the group will be created

e.g; **sudo delgroup section-b**

The above command will simply delete the group named section-b and the member users of this group will no longer be a part of this group. The group entry will also be removed from the **/etc/group** file

Note : root user doesn’t need to use sudo with its privileged commands, it's just a compulsion for the members of the sudo group to use sudo keyword here.

Groups in Linux （ Modifying a group ）

We can make modifications to a group by simply using **groupmod** command

this command is used to make all sort of modifications to a group, so we will have to use appropriate flags/switches and provide appropriate arguments to specify what kind of modification we want to do

Modifications include ; setting password for a group, renaming a group, changing GID for a group, **appending users to a group**

e.g **sudo groupmod -aU ali-hafeez section-b**

Above command will append **ali-hafeez** as a user member of **section-b** group but this group will only be added to the supplementary groups of ali-hafeez and the **primary group of a user can’t be changed with this command.**

Note : root user doesn’t need to use sudo with its privileged commands, it's just a compulsion for the members of the sudo group to use sudo keyword here.

/etc/group

In our directory system, we also have a file named **group** which is located in the /etc/ directory

This file contains entries for all users and system groups and we can even make modifications to the groups by editing this file but again it's not considered a good practice and we have better ways of doing modifications.

Modifying a user account (usermod command)

We can make a lot of modifications to any user account and the command used for carrying out all these modifications is **usermod**

Modifications include ; changing username, changing uid, changing primary group (and the gid) of a user, changing shell type of a user, locking/unlocking a user, adding/removing any supplemental group for the user ….. etc etc

e.g ; **sudo usermod -g section-b ali-hafeez**

above command will forcefully change the **primary group** of ali-hafeez and set it to section-b

Modifying a user account (usermod command ……)

e.g ; **sudo usermod -g section-b ali-hafeez**

above command will forcefully change the **primary group** of ali-hafeez and set it to section-b

e.g ; **sudo usermod -aG section-a ali-hafeez**

above command will add section-a to **supplemental groups** of ali-hafeez and ali-hafeez will become a member of these groups

e.g ; **sudo usermod -s /usr/bin/sh ali-hafeez**

above command will simply change the default shell for **ali-hafeez** to sh

OS Lab

Ownerships & Permissions in Linux

Why Linux is considered so secure?

Among many other reasons, **permission mechanisms** in linux contribute a lot towards its security.

By default configurations, principle of **least privilege** is followed in Linux.

Normal users are usually not allowed to perform highly privileged activities.

A user is not allowed to access directory system of another user

Even members of the sudo group need to add sudo keyword to perform privileged activities and they even need to type their password in the terminal. It adds an extra layer of security to the system

Permissions in Linux

Linux file permissions define which **user** and **group** can read, write, or execute a file or directory.

Each file/directory has permissions assigned to the **owner(user)**, **group**, and **others**, using combinations of **read** (r), **write** (w), and **execute** (x) rights to control access.

Type of Permissions in Linux

Read (r)

Write (w)

Execute (x)

Categories of Users (in context of permissions)

owner (user)

group (a primary group for the file)

others (all other users and groups)

Default ownership

By default, the **creator** of a file/directory is assigned as the owner of that file/directory.

By default, the primary group of the creator is consider the group of a file/directory.

Commands for Modifying permissions

chown (for modifying owner of a file/directory)

chgrp (for modifying group of a file/directory)

chmod (for changing permissions of a file/directory)

chown

e.g **sudo chown ali-hafeez file.txt**

Above command will change the owner of file.txt and set ali-hafeez as owner of that file.

e.g **sudo chown -R ali-hafeez folder**

Above command will change the owner of **folder** directory and its contents recursively and set ali-hafeez as the owner.

Note : root user doesn’t need to use sudo with its privileged commands, it's just a compulsion for the members of the sudo group to use sudo keyword here.

chgrp

e.g **sudo chgrp section-c file.txt**

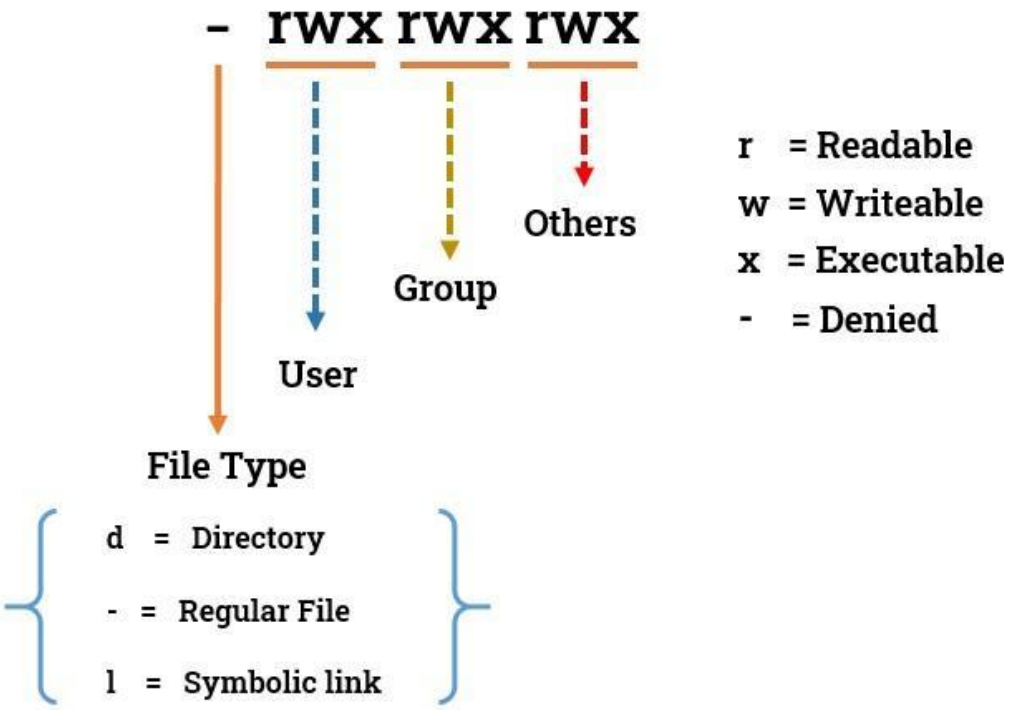
Above command will change the group of file.txt and set **section-c** as group for that file.

e.g **sudo chgrp -R section-c folder**

Above command will change the group of **folder** directory and its contents recursively and set **section-c** as the group.

Note : root user doesn’t need to use sudo with its privileged commands, it's just a compulsion for the members of the sudo group to use sudo keyword here.

Permissions in Linux



111
rwx

chmod 3 digit number (mode bits)

r (4)	w (2)	x (1)	Decimal Digit
0	0	0	0
0	0	1	1
0	1	0	2
0	1	1	3
1	0	0	4
1	0	1	5
1	1	0	6
1	1	1	7

chmod

e.g ; chmod u=rw,g=r,o=r file.txt

Above command will set the permissions of file.txt such that the user will be able to read and write , members of the group of file.txt will be able to read only, and all other users will be able to read only as well.

e.g ; chmod u-x new-file.txt

Above command will make only these changings to the permission that it'll disable the permission of the user(owner) of this file to execute this file. Rest of existing permissions will remain the same

e.g ; chmod g+r new-file.txt

Above command will make changings to the permission such that it'll only enable the permission of the members of the **group** of this file to read this file. Resgt of existing permissions will remain the same.

chmod

chmod 750 file.txt

Above command will simply set the permissions of file.txt by using mod bits.

First digit will decide permissions for 1st category user(owner), 2nd digit will decide the permissions for 2nd category group of that file, and 3rd digit will decide permissions for the 3rd category (all other users and groups)

chmod 3 digit number (Usage)

We simply have to give 1 digit (decimal value) for each category

e.g; **chmod -R 754 folder**

The above command sets permissions as follows;

user=rwx, group=r-x, others=r--

since it's a directory, we used -R to define that we wants our changes to reflect recursively in all sub-directories and files present in the folder

Access Control Lists (ACL)

It’s a mechanism which helps us set permissions exclusively for some particular users or groups (which otherwise belongs to the “others” category in our permissions)

getfacl

e.g ; getfacl filename

setfacl

e.g ; setfacl -m u:ali:rwx filename

umask

This command tells us about the default permissions which will be applied when a file is created by a user

We can make changes to the default permissions temporarily by simply giving this command arguments, these changes will be applicable as far as we don’t terminate our terminal session.

To make permanent changes, we need to modify it in .**bashrc** configuration file.